

Linux Basics — Cheat Sheet

Absolute beginner reference. Everything you need to move around a Linux/macOS terminal and not get lost.

The shell in one idea

You type a **command**, press **Enter**, it runs, it prints a result, you get a fresh prompt. A command is usually `program [options] [arguments]`, e.g. `ls -l /etc`. Options start with `-` (short) or `--` (long).

Where am I? Paths

Symbol	Means
<code>/</code>	the root of the whole filesystem (top of everything)
<code>~</code>	your home directory (<code>/home/you</code> or <code>/Users/you</code>)
<code>.</code>	the current directory
<code>..</code>	the parent directory (one level up)
<code>-</code>	(with <code>cd</code>) the previous directory you were in

Absolute path starts at `/` → `/etc/hosts` (same everywhere). **Relative path** starts from where you are → `dept/hr.html`.

Moving and looking around

Command	What it does	Example
<code>pwd</code>	print working directory (where am I?)	<code>pwd</code>
<code>ls</code>	list files	<code>ls</code>
<code>ls -l</code>	long list: permissions, size, date	<code>ls -l</code>
<code>ls -la</code>	long list including hidden (dotfiles)	<code>ls -la ~</code>
<code>ls -lh</code>	human-readable sizes (K, M, G)	<code>ls -lh</code>
<code>cd DIR</code>	change directory	<code>cd /var/log</code>
<code>cd</code> or <code>cd ~</code>	go home	<code>cd</code>
<code>cd ..</code>	go up one level	<code>cd ..</code>
<code>cd -</code>	go back to previous dir	<code>cd -</code>
<code>tree</code>	show the folder tree	<code>tree -L 2</code>

Creating, copying, moving, deleting

Command	What it does	Example
<code>mkdir DIR</code>	make a directory	<code>mkdir project</code>
<code>mkdir -p a/b/c</code>	make nested dirs in one go	<code>mkdir -p case/notes</code>
<code>touch FILE</code>	create an empty file / update its date	<code>touch todo.txt</code>
<code>cp SRC DST</code>	copy a file	<code>cp a.txt b.txt</code>
<code>cp -r SRC DST</code>	copy a directory (recursive)	<code>cp -r dir/ backup/</code>
<code>mv SRC DST</code>	move or rename	<code>mv old.txt new.txt</code>
<code>rm FILE</code>	delete a file (no trash, forever)	<code>rm junk.log</code>
<code>rm -r DIR</code>	delete a directory and its contents	<code>rm -r tmp/</code>
<code>rmdir DIR</code>	delete an empty directory	<code>rmdir empty/</code>

Danger. `rm` does not ask and does not undo. Double-check before `rm -r`. Never run `rm -rf /`.

Reading files

Command	What it does	Example
<code>cat FILE</code>	print the whole file	<code>cat notes.md</code>
<code>less FILE</code>	scroll a file (q to quit, / to search)	<code>less big.log</code>
<code>head FILE</code>	first 10 lines	<code>head -n 20 data.csv</code>
<code>tail FILE</code>	last 10 lines	<code>tail -n 50 app.log</code>
<code>tail -f FILE</code>	follow a file live (logs)	<code>tail -f app.log</code>
<code>wc -l FILE</code>	count lines	<code>wc -l users.csv</code>

Finding things

Command	What it does	Example
<code>find PATH -name PAT</code>	find files by name	<code>find . -name "*.jpg"</code>
<code>find PATH -type d</code>	find directories	<code>find /etc -type d</code>
<code>grep PAT FILE</code>	find lines containing a pattern	<code>grep "error" app.log</code>
<code>grep -r PAT DIR</code>	search recursively in a folder	<code>grep -r "TODO" src/</code>
<code>grep -i PAT FILE</code>	case-insensitive	<code>grep -i admin users.csv</code>
<code>which CMD</code>	where is this program?	<code>which python3</code>

Redirection and pipes (the superpower)

Symbol	Means	Example
<code>\ </code>	send output of one command into another	<code>cat f \ grep x</code>
<code>></code>	write output to a file (overwrite)	<code>ls > list.txt</code>
<code>>></code>	append output to a file	<code>echo hi >> log.txt</code>
<code><</code>	read input from a file	<code>sort < names.txt</code>
<code>2></code>	redirect errors only	<code>cmd 2> errors.txt</code>
<code>&></code>	redirect output and errors	<code>cmd &> all.txt</code>

Chained example: `cat access.log | grep 404 | wc -l` → count the 404 errors.

Wildcards (globs)

Glob	Matches	Example
<code>*</code>	any characters	<code>ls *.txt</code>
<code>?</code>	exactly one character	<code>ls file?.log</code>
<code>[abc]</code>	one of a, b, c	<code>ls img[12].png</code>
<code>{a,b}</code>	a or b	<code>cp file.{txt,bak}</code>

Permissions (reading `ls -l`)

```
-rwxr-xr-- 1 user group 4096 Sep 16 10:00 script.sh
```

- First char: `-` file, `d` directory, `l` link.
- Then three groups of `rwx` = **owner**, **group**, **others**. `r` read, `w` write, `x` execute.

Command	What it does	Example
<code>chmod +x FILE</code>	make a script executable	<code>chmod +x run.sh</code>
<code>chmod 644 FILE</code>	rw for owner, r for others	<code>chmod 644 note.txt</code>
<code>chmod 755 FILE</code>	rwX owner, rx others	<code>chmod 755 run.sh</code>
<code>chown USER FILE</code>	change owner (often needs sudo)	<code>sudo chown me file</code>
<code>sudo CMD</code>	run a command as administrator	<code>sudo apt update</code>

Processes

Command	What it does
<code>ps aux</code>	list all running processes
<code>top / htop</code>	live process monitor (q to quit)
<code>cmd &</code>	run a command in the background
<code>jobs</code>	list background jobs of this shell
<code>fg / bg</code>	bring a job to foreground / background
<code>kill PID</code>	stop a process by its number
<code>kill -9 PID</code>	force-kill (last resort)

Archives & compression

Command	What it does
<code>tar -czf out.tgz DIR/</code>	create a gzip tarball
<code>tar -xzf out.tgz</code>	extract a gzip tarball
<code>tar -tzf out.tgz</code>	list contents without extracting
<code>zip -r out.zip DIR/</code>	create a zip
<code>unzip out.zip</code>	extract a zip
<code>gzip FILE / gunzip FILE.gz</code>	compress / decompress one file

Network & system (quick)

Command	What it does
<code>ip a (or ifconfig)</code>	show your network interfaces / IPs
<code>ping HOST</code>	test reachability (Ctrl+C to stop)
<code>curl -s URL</code>	fetch a URL (silent)
<code>wget URL</code>	download a file
<code>df -h</code>	disk space, human-readable
<code>du -sh DIR</code>	size of a folder
<code>uname -a</code>	kernel / system info
<code>whoami</code>	your username

Package management (Debian/Kali)

Command	What it does
<code>sudo apt update</code>	refresh the package list
<code>sudo apt install NAME</code>	install a program
<code>sudo apt remove NAME</code>	uninstall
<code>apt search TERM</code>	search for a package

Getting help

Command	What it does
<code>man CMD</code>	full manual (q to quit, / to search)
<code>CMD --help</code>	quick usage summary
<code>tldr CMD</code>	practical examples (install: <code>pip install tldr</code>)
<code>type CMD</code>	is it a program, alias or builtin?

Keyboard shortcuts (save minutes every day)

Keys	Action
<code>Tab</code>	auto-complete a command or filename
<code>↑ / ↓</code>	previous / next command in history
<code>Ctrl+R</code>	search command history (type to filter)
<code>Ctrl+C</code>	stop the running command
<code>Ctrl+D</code>	end of input / logout
<code>Ctrl+L</code>	clear the screen (same as <code>clear</code>)
<code>Ctrl+A / Ctrl+E</code>	jump to start / end of line
<code>Ctrl+U / Ctrl+K</code>	delete to start / end of line

Golden rules. Quote paths with spaces (`cd "My Folder"`). Read before you `rm` . When stuck, `man` it or add `--help` .